

utiliser impérativement les feuilles quadrillées fournies pour les schémas simplifiés

Une réponse non justifiée sera considérée comme fausse

On souhaite étudier la performance d'exécution d'une fonction de tri sur une réalisation superscalaire de l'architecture Mips. Il s'agit de la réalisation à 2 pipelines appelée SS2 étudiée en cours. On considère deux versions différentes de la même fonction.

On considère la fonction **Partition**. Cette fonction prend en entrée un tableau d'entiers **src** et le recopie dans un autre tableau **trg** en séparant les plus petits éléments des plus grands. En parcourant le tableau **src**, chaque élément est comparé au premier élément du tableau. S'il est plus petit que celui-ci, il est placé au début du tableau **trg** sinon à la fin du tableau **trg**.

<pre>unsigned int Partition (int *src, int *trg, unsigned int size) { unsigned int i = 0; unsigned int idx_d = 0; unsigned int idx_f = size - 1; int pivot = 0; pivot = src [0]; for (i=size ; i!=0 ; i--)</pre>	<pre>{ if (src [i-1] < pivot) { trg [idx_d] = src [i-1]; idx_d ++; } else { trg [idx_f] = src [i-1]; idx_f --; } } return (idx_d); }</pre>
---	---

Dans un premier temps, on suppose que le système mémoire est parfait.

Un premier compilateur a produit une première version de code assembleur pour la boucle principale de cette fonction.

On suppose qu'à l'entrée de la boucle le registre R4 contient l'adresse du tableau **src**, R5 l'adresse de fin du tableau **src** (l'adresse qui suit immédiatement celle de la dernière case du tableau), R6 l'adresse du tableau **trg**, et R7 l'adresse de la dernière case du tableau **trg**. On suppose que R8 contient la valeur du **pivot**.

Version 1 :

```
_For : Lw      r9 , -4 (r5 )      ; src [i-1]
      Slt      r10, r9 , r8      ; src [i-1] < pivot ?
      Beq      r10, r0 , _Else
      Nop
      Sw      r9 , 0 (r6 )
      Addiu   r6 , r6 , 4
      J       _Endif
      Nop
_Else : Sw      r9 , 0 (r7 )
      Addiu   r7 , r7 , -4
_Endif: Addiu   r5 , r5 , -4
      Bne     r4 , r5 , _For
      Nop
```

On suppose que l'étiquette **For** est placée sur une adresse alignée (multiple de 8) et qu'à l'entrée de la boucle, le buffer d'instructions est vide.

- 1- Analyser l'exécution de la boucle à l'aide d'un schéma simplifié sur SS2 dans le cas où le branchement échoue et dans le cas où le branchement réussit. Combien de cycles sont-ils nécessaires pour l'exécution d'une itération dans chaque cas ?
- 2- Optimiser le code à l'aide de la technique *Software pipeline* de manière à éviter au maximum les cycles perdus (expliquer votre démarche). Indiquer clairement le nombre d'étages et les instructions contenues dans chaque étage.

Un second compilateur a produit une autre version pour le code de la boucle. On suppose de nouveau que l'étiquette **For** est placée sur une adresse alignée (multiple de 8) et qu'à l'entrée de la boucle, le buffer d'instructions est vide.

Version 2 :

```
_For : Lw      r9 , -4 (r5 )      ; src [i-1]
        Slt    r10, r9 , r8      ; src [i-1] < pivot ?
        Sll    r10, r10, 2
        Sw     r9 , 0 (r6 )
        Sw     r9 , 0 (r7 )
        Addiu  r7 , r7 , -4
        Addu   r7 , r7 , r10
        Addu   r6 , r6 , r10

        Addiu  r5 , r5 , -4
        Bne    r4 , r5 , _For
        Nop
```

- 3- Analyser l'exécution de la boucle à l'aide d'un schéma simplifié sur SS2. Combien de cycles sont-ils nécessaires pour l'exécution d'une itération ?
- 4- Optimiser le code à l'aide de la technique *Software pipeline* de manière à éviter au maximum les cycles perdus (expliquer votre démarche). Indiquer clairement le nombre d'étages et les instructions contenues dans chaque étage.

On souhaite étudier maintenant l'influence du système mémoire sur la performance des deux versions. Dans le reste du sujet, on considère le **code original** des deux versions.

On suppose que le tableau **src** contient 256 entiers. Les adresses des deux tableaux **src** et **trg** sont alignées (multiples de 1024).

On évalue la performance d'exécution des deux codes sur différentes plateformes matérielles. Sur toutes les plateformes, le processeur est relié à un cache de données et à un cache d'instructions séparés. Le cache d'instructions est parfait (taux de miss = 0).

On suppose qu'au début de l'exécution de la fonction le cache de données est vide.

Le cache de données a une capacité de 1 Koctets. C'est un cache à correspondance directe (*Direct Mapping*) et un bloc du cache contient 64 octets. Le temps d'accès au cache est de 1 cycle en cas de hit.

Le cache de données est relié à la mémoire primaire à travers un bus. Ce bus permet d'effectuer une transaction à la fois. A chaque cycle, on peut transférer 4 octets (1 mot) sur le bus. Pour initialiser le transfert, il faut, en moyenne, 6 cycles. Ainsi, en cas de Miss dans le cache, il faut 1 cycle pour détecter le Miss, puis 22 cycles ($6 + 1 \times \frac{64}{4}$) pour transférer le bloc manquant. Après le chargement du bloc, il faut encore 1 cycle pour mettre à jour le cache et 1 cycle supplémentaire pour répondre au processeur.

Sur la première plateforme, on utilise un cache de données **Write Through** qui dispose d'un buffer d'écritures postées de 2 places. Pour une écriture, si le bloc à écrire n'est pas présent, il n'est pas chargé dans le cache. S'il y a au moins une place libre dans le buffer, une écriture coûte 1 cycle, le temps nécessaire pour écrire dans le buffer. Chaque requête d'écriture postée est réalisée par une transaction sur le bus. Une requête d'écriture postée présente dans le buffer est supprimée à la fin de la transaction sur le bus ce qui nécessite en moyenne $7 \left(6 + 1 \times \frac{4}{4}\right)$ cycles.

- 5- Quelle est l'utilité de disposer de 2 places dans le buffer d'écritures postées ?
- 6- Pour la Version 1, combien de Miss se produisent-ils lors de l'exécution de la boucle ?
Calculer le nombre moyen de cycles supplémentaires par itération dus aux lectures.
Calculer le nombre moyen de cycles supplémentaires par itération dus aux écritures.
En déduire le nombre moyen de cycles par itération dans le cas où le branchement réussit et dans le cas où il échoue.
- 7- Pour la Version 2, combien de Miss se produisent-ils lors de l'exécution de la boucle ?
Calculer le nombre moyen de cycles supplémentaires par itération dus aux lectures.
Calculer le nombre moyen de cycles supplémentaires par itération dus aux écritures.
En déduire le nombre moyen de cycles par itération.

Sur la seconde plateforme, on utilise un cache de données **Write Back**. Lors d'une écriture, si le bloc à écrire n'est pas présent, le cache commence par le charger depuis la mémoire puis, le modifie.

On suppose que tous les éléments du tableau **src** sont plus **grands** que le pivot.

- 8- Pour la Version 1, combien de Miss se produisent-ils lors de l'exécution de la boucle (distinguer 2 cas) ?
Calculer le nombre moyen de cycles supplémentaires par itération dus aux lectures.
Calculer le nombre moyen de cycles supplémentaires par itération dus aux écritures.
En déduire le nombre moyen de cycles par itération.
- 9- Mêmes questions pour la Version 2.

On suppose que tous les éléments du tableau **src** sont plus **petits** que le pivot.

- 10- Pour la Version 1, combien de Miss se produisent-ils lors de l'exécution de la boucle (distinguer 2 cas) ?
Calculer le nombre moyen de cycles supplémentaires par itération dus aux lectures.
Calculer le nombre moyen de cycles supplémentaires par itération dus aux écritures.
En déduire le nombre moyen de cycles par itération.
- 11- Mêmes questions pour la Version 2.